

Distributed Autonomous Building Control System

System-Level Cyber-Physical Architecture, IoT Simulation, Data Pipelines & Autonomous Agents

Course Code: D7065E / 17073

Target Language: Go (Golang) / Containers

Group Structure: Pairs (2 Persons)

Core Framework: BuildSim & Docker

Evaluation: System Architecture & Oral Defense

Checkpoint: Week 3 Approval Required

1. Executive Summary & Overview

In this project, you will design, engineer, and evaluate a **distributed autonomous building control system**. The system models a real-world cyber-physical environment comprising physical dynamics, IoT sensors and actuators, streaming data pipelines, autonomous decision agents, and interactive visual monitoring.

Rather than executing as a single monolithic script, the system must consist of **multiple independent, containerized microservices** running locally alongside **BuildSim**—a provided 3D building simulator and REST API engine. The primary objective is to build a robust, fault-tolerant systems architecture capable of closed-loop physical control, real-time telemetry processing, and autonomous decision-making.

System-Level Cyber-Physical Approach

Testing autonomous control systems on physical infrastructure is costly, dangerous, and hard to reproduce. This course utilizes a *system-level simulation* approach. The physical dynamics, sensor responses, actuator effects, data pipelines, and network protocols are engineered as a complete distributed deployment, ensuring code and architectural patterns remain directly transferable to real-world smart buildings.

2. Key System Architecture & Flow

All microservices run locally on the student's machine and interact through standard REST APIs, WebSockets, or messaging protocols. BuildSim serves as the ground-truth authority for the building layout and state.

1. Execution Workflow

- **Start BuildSim:** Launch executable `./buildsim start --port 9090`.
- **Deploy Microservices:** Execute `docker compose up` to spin up containers.
- **Physical Dynamics Loop:** The simulator updates environmental variables (heat, CO2, smoke).
- **Sensor Ingestion:** Sensor services push readings to BuildSim REST APIs & storage.

2. Autonomous Control Loop

- **AI Agents:** Query sensor metrics, reason via rule logic or LLM, and issue actuator commands.
- **Actuator Feedback:** Actuators modify physical state in BuildSim, altering simulator inputs.
- **3D Dashboard:** Streams real-time state via WebSockets for visual monitoring.
- **GPU LLM Server:** External Gemma 4 model via OpenAI-compatible endpoint.

Core Architectural Principles

- **Shared State Authority:** BuildSim maintains current state (sensors, actuators, active sessions).
- **Service Independence:** Components are decoupled processes. If a process crashes and restarts, it must automatically re-register with BuildSim and seamlessly resume operation.
- **Closed Loop Feedback:** Actuator commands must directly impact physical environmental simulation, which subsequently alters future sensor observations.

3. Detailed Project Requirements

A. Architecture & System Design Specification

System architecture must be fully documented and approved before implementation begins (Week 3 Checkpoint).

- **Architecture Document:** High-level design diagrams detailing component boundaries, communication protocols (Pub/Sub, REST, WebSockets), data flow, storage strategies, fault resilience, and horizontal scaling. *Every single design choice must be rigorously justified* with trade-off analysis.
- **Design Specification:** Explicit API contracts, schema definitions, data models, AI agent reasoning design, and sensor/actuator state machines.
- **Test Plan:** Verification matrix covering unit tests, integration pipelines, AI validation, and rigorous fault-injection failure scenarios.

B. Physical Simulator & Environmental Dynamics

You must construct a physical simulator that models realistic environmental physics and feeds sensor processes. Sensor data must not be random noise; it must possess clear structural patterns (trends, delays, correlations).

Scenario Type	Physical Behavior & Dynamics	Actuator Feedback Effect
Fire & Smoke Propagation	Smoke concentration grows exponentially, spreads to adjacent corridor zones; temperature rises with thermal intensity.	Sprinkler activation reduces ambient temperature and suppresses smoke accumulation.
HVAC & Climate Control	Room temp is a function of external weather, thermal mass, occupancy body heat, and insulation loss.	HVAC heating/cooling alters room temperature gradually based on thermal inertia equations.
Air Quality (IAQ)	CO2 increases by ~40 ppm/person/hour in closed rooms based on occupancy count.	Increasing ventilation fan RPM or opening automated dampers rapidly attenuates CO2 levels.
Access Control & Security	Arrival peaks at 07:00–09:00, lunch traffic at 11:30–13:00, departure at 16:00–18:00, empty on weekends.	Door unlocking/locking based on identity validation and contextual occupancy policies.
Predictive Maintenance	Mechanical pump/bearing vibration increases gradually over weeks with intermittent harmonic spikes.	Load rebalancing or automated throttle adjustments mitigate severe mechanical degradation.

Occupancy Drivers: Simulator profiles must model temporal occupancy dynamics (weekdays vs. weekends, 1-2 hour meeting room bookings, corridor transients, and seasonal variations). Occupancy directly influences heat, CO2, lighting, and network load.

C. Service Implementation & Containerization

- Every sensor, gateway, and actuator must run as an independent Unix process in a standalone Docker container.

- Services must support isolated lifecycle operations (`docker start` , `docker stop` , `docker restart`).
- **Recommended Language: Go (Golang)** is strongly recommended due to its efficient static compilation, light memory footprint, and native concurrency support. Python, Rust, or Java are allowed if strongly justified.

D. Autonomous AI & Decision-Making Integration

At least one independently deployable component must autonomously monitor sensor feeds and issue actuator controls. Sophisticated AI is not required for a high grade; architectural integration and reliability are key.

- **Numerical ML** Anomaly detection, timeseries prediction, or equipment failure prediction.
- **LLM Reasoning** Regulatory compliance checking against building codes (BBR, AFS 2020:1), emergency protocols, or equipment manuals using local GPU LLM inference.
- **Hybrid System** Combining fast ML anomaly detection with an LLM agent that interprets complex regulatory contexts.

E. Data Engineering & Broker Pipeline

Direct sensor-to-agent REST calls are prohibited for primary telemetry. System must implement a structured pipeline:

- **Message Broker:** Decouple producers and consumers using MQTT, Apache Kafka, or Redis.
- **Data Storage:** Persist timeseries telemetry in InfluxDB, TimescaleDB, DuckDB, or Parquet files.
- **Orchestration Architecture:** Optionally leverage SOA frameworks (Arrowhead), Event Sourcing, Stream Processing, or **ColonyOS** for edge-cloud orchestration.

4. Public Reference Datasets for Simulation

Students may utilize physics-based models, data replay, hybrid event injection, or generative models using the following public benchmarks:

Dataset Name	Description & Sensor Types	Primary Application
ASHRAE Energy Prediction	Building energy consumption, hourly weather, ambient conditions	HVAC & Thermal Simulation
Building Data Genome Project 2	3,053 energy meters from 1,636 buildings over 2 complete years	Energy Consumption Modeling
Environmental Sensor Data	132k readings: temperature, humidity, CO2, PM2.5 from IoT nodes	Air Quality & Comfort Control
Occupancy Detection Dataset	Light, temperature, humidity, CO2 tagged with ground-truth occupancy labels	Occupancy & IAQ Modeling
Ecobee Thermostat Data	Indoor/outdoor thermal dynamics, setpoints, HVAC operational cycles	Thermostat Control Logic
Smart Building Dataset	Multi-sensor suite across smart office floorplans	Multi-zone Microclimate
Pump & Bearing Datasets	NASA vibration, PRONOSTIA accelerometer, and industrial pump telemetry	Predictive Maintenance ML

5. Workflow, Assessment & Examination

- 1
- Use Case Selection:** Choose a standard building control domain (Fire, HVAC, Security, IAQ, Maintenance) or propose a custom scenario.

- 2 **Architecture & Design Specification:** Formalize component diagrams, data contracts, and trade-off justifications.
- 3 **Test Plan Definition:** Specify verification methods and concrete fault-injection experiments.
- 4 **Design Approval (Week 3 Checkpoint):** Present and defend design specification before writing core code.
- 5 **Containerized Implementation:** Develop Go microservices, physical simulator, data pipeline, and AI logic.
- 6 **System Demonstration & Defense:** Live demonstration of live closed-loop control and resilience under simulated failure.

Oral Examination & Grading Policy

The oral exam evaluates individual systems understanding. Each student must be able to draw the entire end-to-end architecture on a whiteboard from memory and explain every design decision, data pipeline, and failure recovery mechanism. Group members are examined individually and must demonstrate full understanding beyond their personal implementation tasks.